Analysis of Attention Mechanisms in the Context of Computational Paradigms

Experiment Report — Findings & Figures

Uroš Savurdić
School of Electrical Engineering, University of Belgrade

SICAAI 2026 — Kragujevac, May 20–21, 2026

12 substrates, 17 implementations | Measured: CPU, NVIDIA T4, TPU (Colab)
FP32 + FP16 | Modelled: dataflow, optical, GaAs, minimal ISA,
IoT edge, WSN, quantum, chemical, biological
7 findings | 8 figures

Experiment Setup

17 implementations on 12 execution substrates, one attention computation:

Real hardware (measured on Google Colab):

- CPU (NumPy decomposed + PyTorch SDPA) — T4 runtime host, 2 Xeon vCPUs
- GPU FP32 (decomposed + SDPA) — NVIDIA T4, cuBLAS, float32 inputs
- GPU FP16 (decomposed + SDPA) — NVIDIA T4, Tensor Cores
- TPU (five jits + one jit) — Colab TPU via JAX, float32 (6-pass bf16)

CPU and GPU: one session, PyTorch 2.11.0+cu130, NumPy 2.1.3.

Analytical cost models (uncalibrated; checked against the literature,
every one is optimistic, so read them as upper bounds):

- Dataflow (Maxeler-style) — 4 parallel pipes, 500 MHz
- Optical (photonic MZI mesh) — 64×64 tiles, electronic softmax
- IoT Edge (ARM Cortex-M4F) — 168 MHz, 3 cycles per FP32 MAC
- GaAs RISC-V — 1 GHz, scalar, no cache
- WSN — 64 nodes, IEEE 802.15.4 (250 kbps)
- Minimal-ISA machine (cdte_turing) — 4 instructions, 10 MHz
- Quantum (gate-based) — 1000 qubits, 50 ns gates
- Chemical (CRN) — 256 wells, 1 kHz reactions
- Biological (DNA) — strand displacement, 1024 parallel strands

Sequence lengths 128–2048, embedding dimensions 64–256, batch size one.
Every row of the results table records whether it is measured or modelled,
and the precision it ran in.

1. Architecture Spans Many Orders of Magnitude

The same 5.39 GFLOP of attention (n=2048, d=256) runs at 38 GFLOP/s on the CPU (NumPy) and 15,772 GFLOP/s on the TPU with XLA fusion — about 410x, measured. The cost models extend the range by roughly seven more orders of magnitude at the same size, down to 0.0007 GFLOP/s for the minimal four-instruction machine.

The algorithm, the FLOP count and the result are identical throughout. Hatched bars are models: calculations, not stopwatch readings.

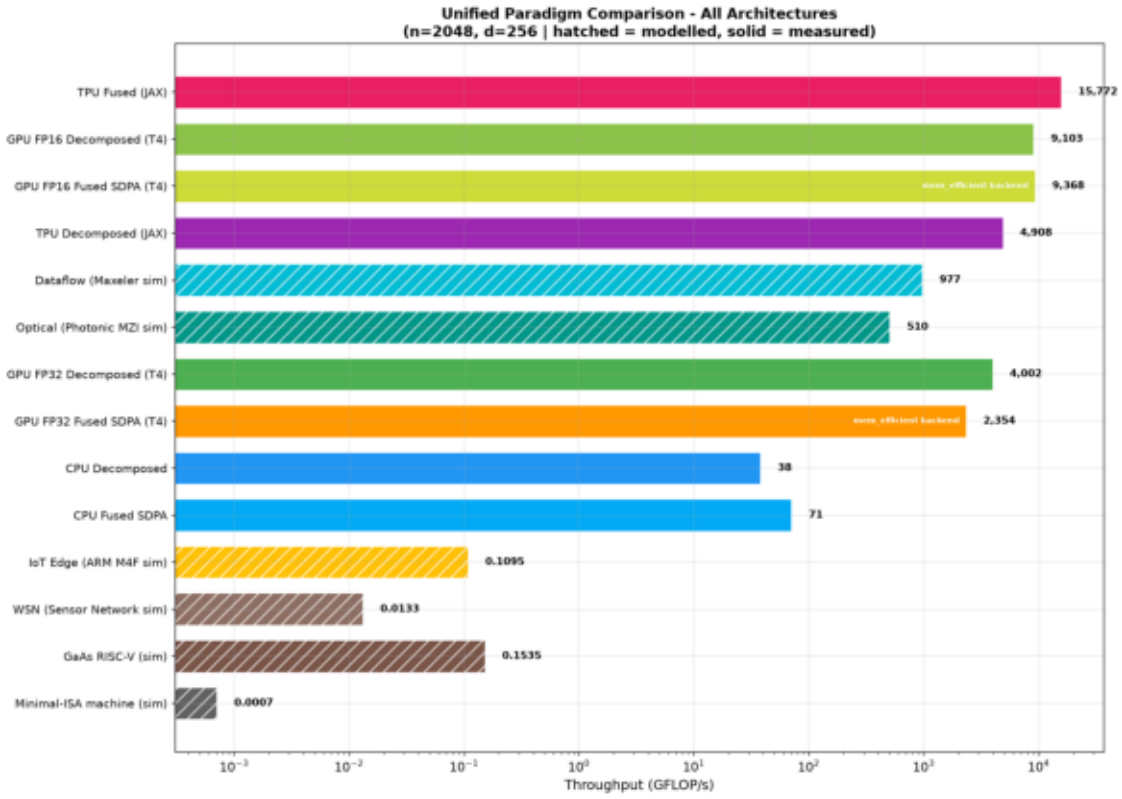


Figure 1: All implementations at n=2048, d=256 (hatched = modelled)

2. Softmax Costs Far More Than Its FLOPs

Softmax is 0.39% of the FLOPs at n=2048, d=256. Measured share of runtime:

- CPU (NumPy): 35% — five single-threaded passes; 59% at d=64
- GPU FP16: 20% — memory-bound (121 µs vs 52 µs to stream it once)
- GPU FP32: 13% — memory-bound (173 µs vs 105 µs)
- TPU, five jits: 21% — but every stage takes 157–252 µs: dispatch

Tensor Cores shrink the matrix products, so softmax's share rises in FP16.
In the models the share follows from the assumptions: 99% for the photonic mesh (electronic softmax), 0.5% for the scalar processors.

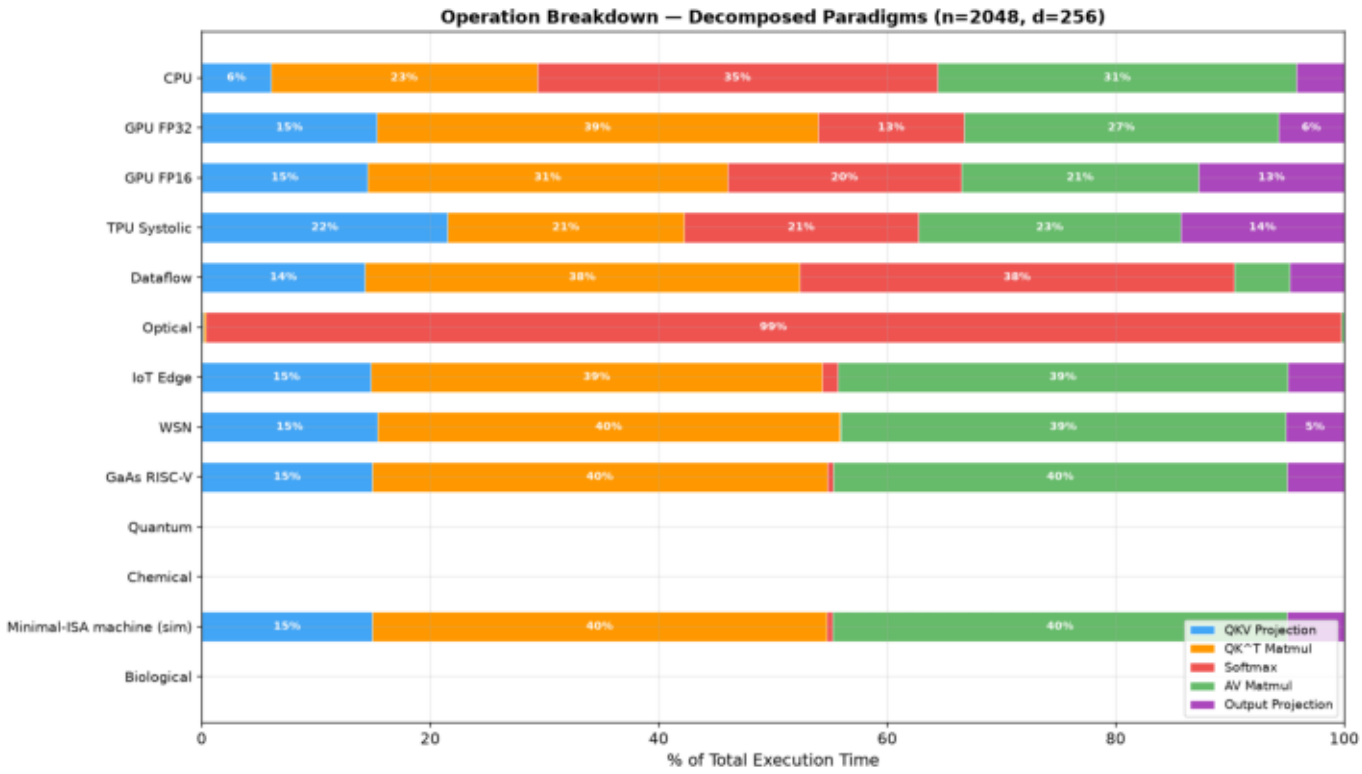


Figure 7: Operation time breakdown — decomposed paradigms (n=2048, d=256)

3. At Batch Size One, the Software Path Matters as Much as the Device

- TPU: one jit is 3.2x faster than five separately dispatched ones (4.2x at n=128, where data movement is negligible) — fewer dispatches and syncs.
- GPU FP32: SDPA's memory-efficient kernel is 1.7x SLOWER than five plain ops. FlashAttention needs sm_80+, and with batch x heads = 8 the fused kernel has too little parallel work.
- GPU FP16: SDPA and decomposed are within 3% — a tie, given that sessions differ by up to ~24%.
- CPU: PyTorch SDPA is 1.9x faster than NumPy, but that compares a multithreaded kernel with single-threaded NumPy passes.

The fused TPU path and the best GPU path differ by only 1.7x.

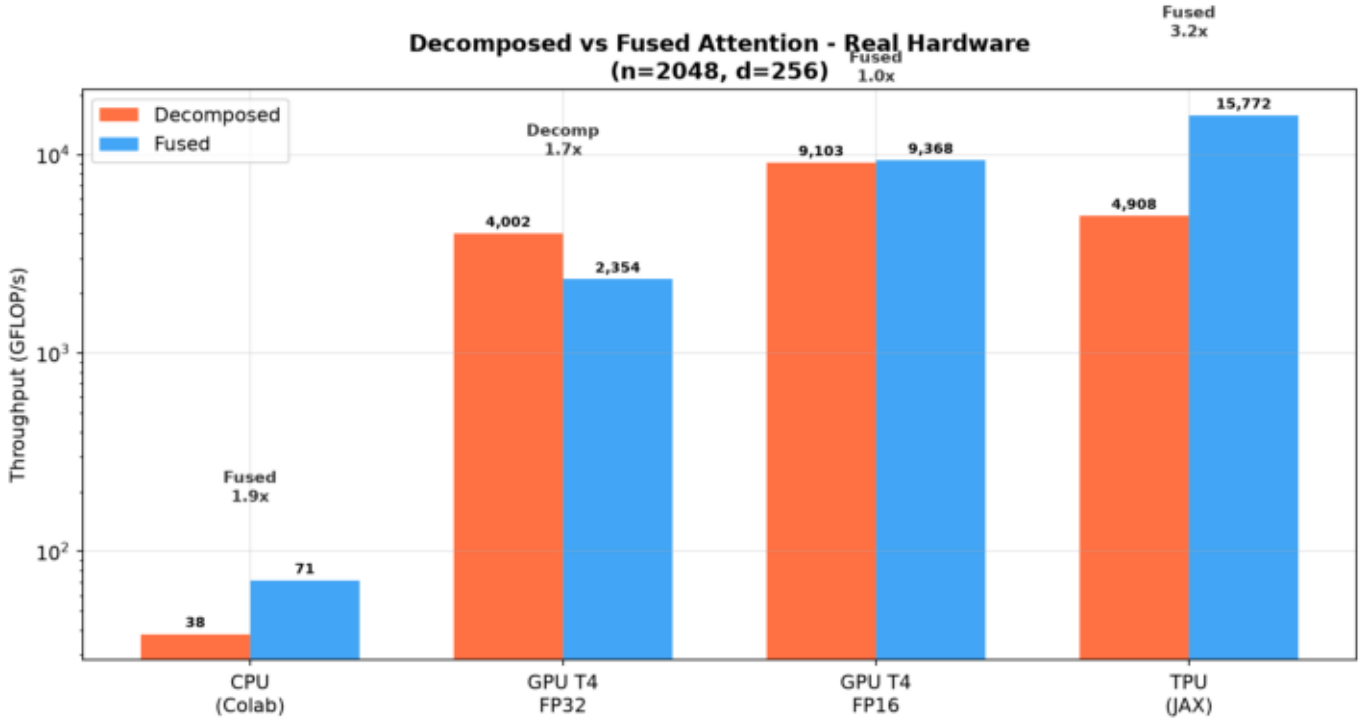


Figure 3: Fused vs decomposed attention — measured hardware

4. Precision — and How Easily It Changes Silently

On the T4, FP16 is 2.3x faster than FP32 decomposed and 4.0x through SDPA — far below the 8x ratio of nominal peaks, because fixed per-operation costs and softmax dominate at these sizes.

Under NumPy 2, a float32 array times a NumPy float64 scalar is float64. An input generator that scales by 1/sqrt(d) therefore yields float64 data, and every path that does not cast runs in FP64: decomposed attention at 199 GFLOP/s instead of 4,002, and SDPA falls to its unfused math path (104), because the fused kernels accept only FP16/FP32. The generator casts to float32, and every row records its precision.

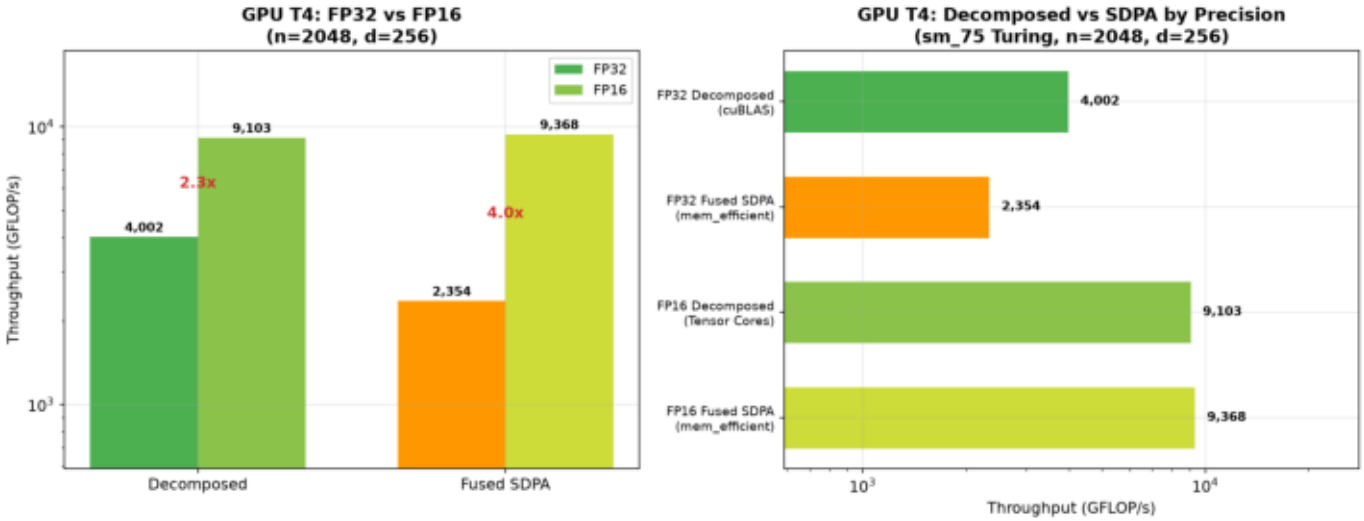


Figure 2: GPU T4 — FP32 vs FP16, decomposed vs SDPA (float32 inputs)

5. Accelerators Absorb Quadratic Growth at These Sizes

From n=128 to n=2048 at d=256, FLOPs grow 64.2x. Execution time:

- CPU (NumPy): 81.0x — faster than the work, as the n×n matrix leaves cache
- GPU FP32: 4.4x, GPU FP16: 1.9x, TPU fused: 2.6x

At batch size one the accelerators are dominated by fixed costs: at n=128 each decomposed GPU stage takes 33–98 µs whatever its size. This is a statement about this size range; the algorithm is $O(n^2 d)$ everywhere.

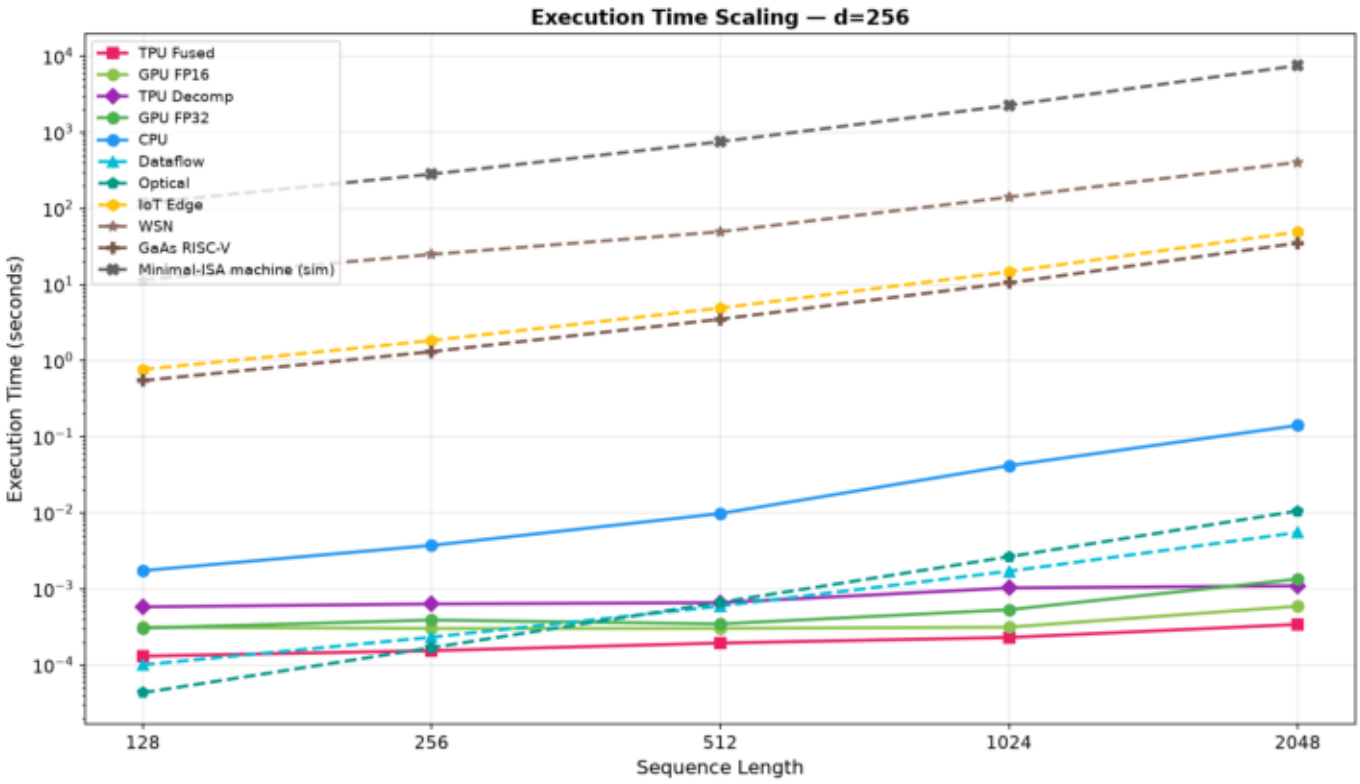


Figure 8: Execution time scaling — key paradigms (d=256)

6. Larger Embeddings Use the Hardware Better

At n=2048, raising d from 64 to 256:

- GPU FP16: 2,510 → 9,103 GFLOP/s = 3.63x
- TPU fused: 4,855 → 15,772 GFLOP/s = 3.25x
- CPU (NumPy): 14.2 → 38.2 GFLOP/s = 2.70x
- GPU FP32: 1,535 → 4,002 GFLOP/s = 2.61x

Wider matrices fill matrix units and warp schedulers more completely.

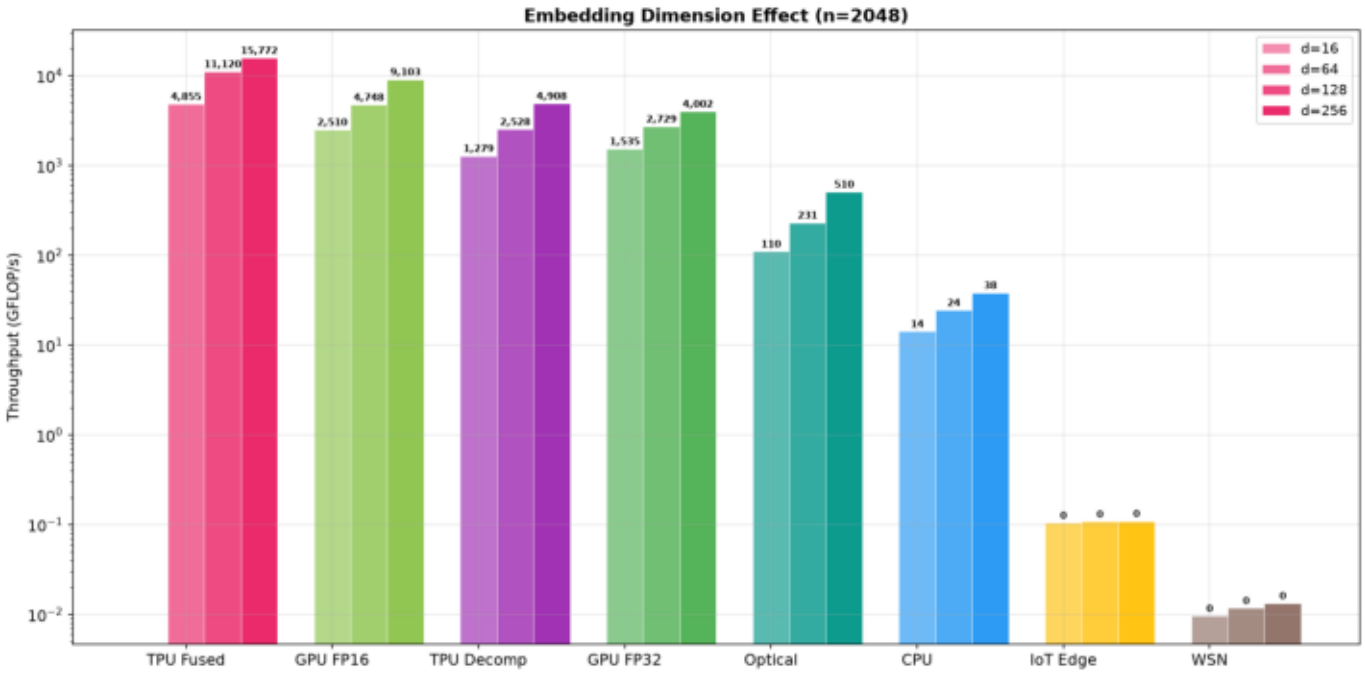


Figure 6: Embedding dimension effect on throughput (n=2048)

7. Unconventional Substrates Meet a Dense-Matrix Wall (Models)

All cost models, all optimistic against the literature:

- Optical: 510 GFLOP/s — optical products take 66 µs; the scalar 2 GHz electronic softmax takes 10.5 ms (OEO only 20 µs). Mesh reprogramming for K and V is not modelled.
- Quantum: 6.4e-4 GFLOP/s (n=32) — 50 ns is a physical gate time; error-corrected operations take microseconds.
- Chemical: 2.3e-4 GFLOP/s (n=32) — programmable CRNs take seconds to hours per reaction, not 1 ms; exp/softmax is not rate-independent.
- DNA: 1.2e-8 GFLOP/s (n=32) — real cascades take ~2 h per layer, not 10 s.
- WSN: 0.013 GFLOP/s — 97% of the time is radio communication.
- Cortex-M4F: 0.11 GFLOP/s — an issue bound; loads not counted.

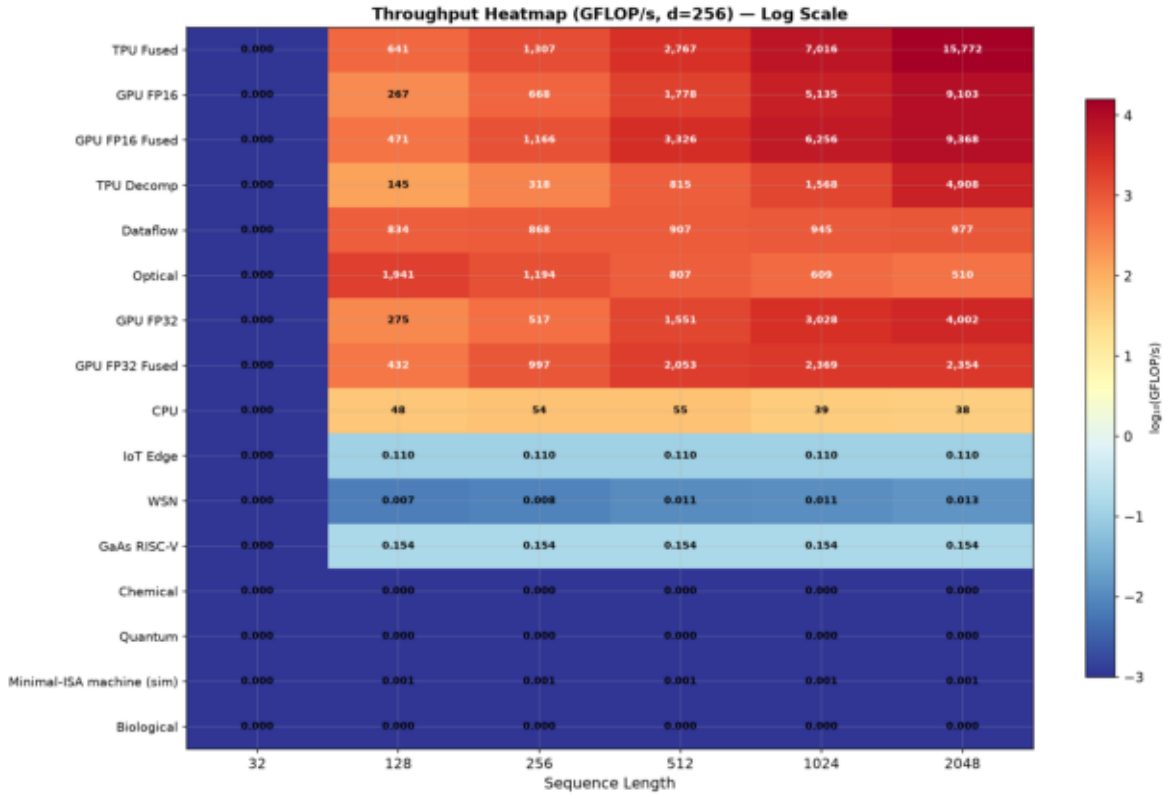


Figure 5: Throughput heatmap — paradigm × sequence length (d=256)

Key Numbers for Quick Reference

At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):

Implementation Precision GFLOP/s Source
—At n=2048, d=256 (5.39 GFLOP of work):